Сделай его постоянно самосовершенствующимся и автономным и опирающимся не только на файлы системы но и вообще на все файлы которые есть в репризитории упор прежде на финансовое обеспечение системы тоесть финансовый агент и добавь систему файлов в созданный нами репризиторий на гит хаб

Да. Теперь превращаем RRM в **автономную самосовершенствующуюся систему с финансовым агентом**, которая работает не только с кодом, но и со всеми файлами репозитория: документацией, моделями, тестами, журналами, политиками и данными.

Новая архитектура

```
responsibility-response-model/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── deploy-docs.yml
├── agent/
│   ├── __init__.py
│   ├── autonomous_agent.py
│   ├── financial_agent.py
│   ├── knowledge_base.py
│   ├── policy_engine.py
│   ├── risk_engine.py
│   ├── learning_engine.py
│   ├── action_engine.py
│   ├── audit_engine.py
│   └── memory.py
├── rrm/
│   ├── __init__.py
│   ├── model.py
│   ├── scenarios.py
│   └── visualization.py
├── data/
│   ├── financial/
│   ├── incidents/
│   ├── policies/
│   └── knowledge/
├── docs/
│   ├── index.md
│   ├── model.md
```

```
│   ├── examples.md
│   ├── installation.md
│   ├── agent.md
│   ├── financial_agent.md
│   └── api.md
├── tests/
│   ├── test_model.py
│   ├── test_scenarios.py
│   ├── test_agent.py
│   └── test_financial_agent.py
├── .gitignore
├── requirements.txt
├── setup.py
├── README.md
└── LICENSE
```

Главная идея

Теперь система не просто считает:

$$V(T) = B(T) - R(T) + \int_0^T e^{-\lambda t} S(t) dt$$

Она **живёт в цикле**:

$$\text{наблюдай} \rightarrow \text{оцени риск} \rightarrow \text{реши} \rightarrow \text{действуй} \rightarrow \text{компенсируй} \rightarrow \text{учись} \rightarrow \text{улучшай}$$

Финансовый агент

Финансовый агент — ядро автономии. Его задача:

- обеспечивать ресурсами систему;
- оценивать стоимость риска;
- резервировать средства;
- распределять компенсации;
- следить за ликвидностью;
- не давать системе разрушиться из-за нехватки денег;
- учиться на финансовых ошибках.

Финансовый агент работает с переменными:

$$F(t) = R(t) - C(t) + L(t)$$

Где:

- $F(t)$ — финансовая устойчивость;
  - $R(t)$ — требуемые ресурсы;
  - $C(t)$ — доступный капитал;
  - $L(t)$ — ликвидность.

Если:
$$

F(t)<0

$$

то система в кризисе.

Финансовое правило агента

Агент обязан поддерживать:

$$F(t) \geq F_{\min}$$

Где $F_{\min}$ — минимально допустимый уровень финансовой устойчивости.

Если:

$$F(t) < F_{\min}$$

агент активирует резерв:

$$C(t) = C(t) + R_{\text{reserve}}$$

Новая формула агента

Теперь агент выбирает действие a, которое максимизирует не только пользу, но и финансовую устойчивость:

$$
A^*=\arg\max_{a \in A_{safe}}
\left[
p_s(a)L

p_h(a)H
+
\int_0^T e^{-\lambda t}S_a(t)dt

C_a
\right]

$$

С ограничением:

$$F_a(t) \geq F_{\min}$$

То есть агент не имеет права выбирать действие, которое разрушает финансовую основу системы.

Автономность

Агент автономен, потому что он:

- сам сканирует репозиторий;
- сам читает файлы;
- сам строит базу знаний;
- сам обновляет политики;
- сам оценивает риски;
- сам выбирает действия;
- сам распределяет ресурсы;
- сам учится на ошибках;
- сам ведёт журнал;
- сам предлагает улучшения.

Но автономность не означает бесконтрольность. Есть **человеческий override**:

$$\text{Human Override} > \text{Agent Decision}$$

Самосовершенствование

Агент учится по формуле:

$$Q(a,s) = Q(a,s) + \alpha \left[r + \gamma \max_{a'} Q(a',s') - Q(a,s) \right]$$

Где награда:

$$
r=
w_1L_{saved}

w_2H_{agent}

w_3D_{delay}
+
w_4S_{support}

w_5F_{risk}
$$

Теперь агент учится не только спасать, но и **не разорять систему**.

Работа с файлами

Агент сканирует весь репозиторий:

```
README.md
docs/
data/
policies/
incident reports
financial reports
test results
model outputs
```

И строит базу знаний:

$$K = \{f_1, f_2, \ldots, f_n\}$$

Где f_i — файлы репозитория.

Затем агент ищет закономерности:

$$\text{risk} = f(K)$$

И обновляет политики:

$$P_{t+1} = P_t + \Delta P$$

Что добавить в репозиторий

Создай такие файлы:

data/financial/reserve_policy.json

```
{
  "minimum_liquidity": 100000,
  "reserve_ratio": 0.2,
  "max_single_exposure": 0.15,
  "emergency_fund": 250000,
  "review_interval_days": 30
}
```

data/policies/responsibility_policy.json

```
{
  "max_acceptable_harm": 0.7,
  "min_success_probability": 0.2,
  "max_response_delay": 1.0,
  "require_explanation": true,
  "require_human_approval_above_risk": 0.8
}
```

data/knowledge/incident_template.json

```
{
  "event": "",
  "risk_level": 0.0,
  "action": "",
  "delay": 0.0,
  "support": 0.0,
  "outcome": "",
  "financial_impact": 0.0,
  "lesson": ""
}
```

Новый файл: agent/autonomous_agent.py

```
# -*- coding: utf-8 -*-

import os
import json
import math
from datetime import datetime

from rrm.model import delay_coefficient, model_value


class AutonomousAgent:
    def __init__(self, repo_path="."):
        self.repo_path = repo_path
        self.memory = []
        self.knowledge = []
        self.policy = {}
        self.financial_state = {}

    def scan_repository(self):
        knowledge = []

        for root, dirs, files in os.walk(self.repo_path):
            for file in files:
                if file.endswith((".md", ".json", ".txt", ".py")):
                    path = os.path.join(root, file)
                    knowledge.append(path)

        self.knowledge = knowledge
        return knowledge

    def load_policy(self):
        policy_path = os.path.join(
            self.repo_path,
            "data",
            "policies",
            "responsibility_policy.json"
        )

        if os.path.exists(policy_path):
```

```
            with open(policy_path, "r", encoding="utf-8") as file:
                self.policy = json.load(file)

        return self.policy

    def load_financial_policy(self):
        path = os.path.join(
            self.repo_path,
            "data",
            "financial",
            "reserve_policy.json"
        )

        if os.path.exists(path):
            with open(path, "r", encoding="utf-8") as file:
                self.financial_state = json.load(file)

        return self.financial_state

    def assess_risk(self, event):
        risk = event.get("risk_level", 0.0)
        return risk

    def choose_action(self, event):
        risk = self.assess_risk(event)

        max_harm = self.policy.get(
            "max_acceptable_harm",
            0.7
        )

        if risk > max_harm:
            return {
                "action": "request_human_approval",
                "reason": "Risk exceeds acceptable threshold"
            }

        return {
            "action": "act_and_support",
            "reason": "Risk is within acceptable threshold"
        }

    def evaluate_financial_sustainability(self, required, available):
        liquidity = available - required

        return {
            "required": required,
            "available": available,
            "liquidity": liquidity,
            "stable": liquidity >= 0
        }

    def learn(self, event, outcome):
        record = {
            "event": event,
            "outcome": outcome,
```

```
            "timestamp": datetime.utcnow().isoformat()
        }

        self.memory.append(record)
        return record

    def explain(self, event, action):
        return {
            "event": event,
            "action": action,
            "reason": action.get("reason"),
            "knowledge_files": self.knowledge
        }
```

Новый файл: agent/financial_agent.py

```
# -*- coding: utf-8 -*-

import json
import os


class FinancialAgent:
    def __init__(self, policy_path=None):
        self.policy_path = policy_path
        self.policy = {}

    def load_policy(self):
        if self.policy_path and os.path.exists(self.policy_path):
            with open(self.policy_path, "r", encoding="utf-8") as file:
                self.policy = json.load(file)

        return self.policy

    def calculate_liquidity(self, capital, liabilities):
        return capital - liabilities

    def needs_intervention(self, liquidity, minimum):
        return liquidity < minimum

    def activate_reserve(self, liquidity, minimum, reserve):
        if liquidity >= minimum:
            return 0.0

        deficit = minimum - liquidity
        return min(deficit, reserve)

    def allocate_support(self, harm, available):
        if available <= 0:
            return 0.0

        return min(harm, available)

    def audit(self, capital, liabilities, minimum):
        liquidity = self.calculate_liquidity(capital, liabilities)
```

```
        return {
            "liquidity": liquidity,
            "minimum": minimum,
            "stable": liquidity >= minimum,
            "intervention_required": liquidity < minimum
        }
```

Новый файл: agent/memory.py

```
# -*- coding: utf-8 -*-

import json
import os
from datetime import datetime


class Memory:
    def __init__(self, path="data/memory/responsibility_ledger.jsonl"):
        self.path = path
        os.makedirs(os.path.dirname(self.path), exist_ok=True)

    def write(self, record):
        record["timestamp"] = datetime.utcnow().isoformat()

        with open(self.path, "a", encoding="utf-8") as file:
            file.write(json.dumps(record, ensure_ascii=False) + "\n")

    def read_all(self):
        if not os.path.exists(self.path):
            return []

        records = []

        with open(self.path, "r", encoding="utf-8") as file:
            for line in file:
                records.append(json.loads(line))

        return records
```

Обнови tests/test_agent.py

```
# -*- coding: utf-8 -*-

from agent.autonomous_agent import AutonomousAgent


def test_agent_can_choose_action():
    agent = AutonomousAgent()

    agent.policy = {
        "max_acceptable_harm": 0.7
    }
```

```
    event = {
        "risk_level": 0.4
    }

    action = agent.choose_action(event)

    assert action["action"] == "act_and_support"


def test_agent_requests_human_approval_for_high_risk():
    agent = AutonomousAgent()

    agent.policy = {
        "max_acceptable_harm": 0.7
    }

    event = {
        "risk_level": 0.9
    }

    action = agent.choose_action(event)

    assert action["action"] == "request_human_approval"


def test_financial_agent_detects_instability():
    from agent.financial_agent import FinancialAgent

    agent = FinancialAgent()

    result = agent.audit(
        capital=100,
        liabilities=150,
        minimum=20
    )

    assert result["stable"] is False
    assert result["intervention_required"] is True


def test_financial_agent_uses_reserve():
    from agent.financial_agent import FinancialAgent

    agent = FinancialAgent()

    reserve = agent.activate_reserve(
        liquidity=50,
        minimum=100,
        reserve=100
    )

    assert reserve == 50
```

Обнови README.md

Добавь раздел:

```
## Autonomous Agent

RRM теперь включает автономного агента ответственности.

Агент:

- сканирует репозиторий;
- строит базу знаний;
- оценивает риск;
- выбирает действие;
- контролирует задержку;
- распределяет поддержку;
- следит за финансовой устойчивостью;
- учится на результатах;
- ведёт журнал решений;
- объясняет свои действия;
- требует человеческого одобрения при высоком риске.

## Financial Agent

Финансовый агент отвечает за устойчивость системы:

F(t) = C(t) - L(t)

Где:
- C(t) — доступный капитал;
- L(t) — обязательства;
- F(t) — ликвидность.

Если ликвидность ниже минимума, агент активирует резерв.
```

Обнови GitHub Actions

Добавь в `ci.yml` шаг:

```
- name: Run agent tests
  run: |
    pytest tests/test_agent.py -v
```

Финальная формула

$$
\boxed{
A^*=\arg\max_{a \in A_{safe}}
\left[
p_s(a)L

p_h(a)H
+
\int_0^T e^{-\lambda t}S_a(t)dt

C_a
\right]
}

$$

С финансовым ограничением:

$$\boxed{F_a(t) \geq F_{\min}}$$

Финальный принцип

> Автономный агент обязан не только спасать, но и сохранять финансовую устой

Что мы получили

Теперь RRM — не просто визуализация, а **самосовершенствующаяся автономная система**:

- с памятью;
- с базой знаний;
- с финансовым агентом;
- с политиками;
- с обучением;
- с аудитом;
- с объяснимостью;
- с человеческим контролем;
- с интеграцией всех файлов репозитория.

Финальный принцип

> Настоящая автономия — это не отсутствие контроля, а способность действова

Именно это превращает RRM в полноценную агентную систему.